Failure (a crashing tool, and a bad description) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

Half 1: first, watch it actually crash — call your failing tool function directly, unwrapped, and let the exception happen. You want to have actually seen the raw traceback once before you hide it, so "catching the error" doesn't feel like skipping a step. Then the key line is `return f"Error: {e}"` inside an `except` block, instead of letting the exception propagate. This is a deliberate design choice — the tool's "successful" return type and its "error" return type should look the same to the code path that reads the result (both are just strings the model can react to), because the model has no other way to see what happened. After catching the error and returning a string, pass that string back into the conversation as the tool's result, and ask the model to respond — check that it reacts sensibly to the error text instead of treating it as a real answer.

Half 2: reuse the two ambiguous tools from the Intermediate ambiguity exercise (or write two new ones) — give one of them a genuinely bad description, run your existing 5-run trial function against it, then edit only the description (not the function body), and run the exact same trial again. Print each run's pick both before and after, and count the split as a ratio (like "4/5 picked tool A") rather than eyeballing it. Write out, for Half 2, exactly what you're going to hold constant between the "before" and "after" runs — the comparison only means something if everything else stays fixed except the description text.

Hint 2 — The plan, and almost the whole thing

Two small pieces to get you unstuck.

Half 1 — send the error back, the same growing-list idea as Doc04's memory exercise:

```
# tool_selection_practice.py — Failure section
messages = [{"role": "user", "content": "Use the tool to divide 10 by 0"}]
response = model_with_tools.invoke(messages)
messages.append(response)
for call in response.tool_calls:
    result = flaky_divide.invoke(call["args"])
    messages.append(
        {"role": "tool", "content": result, "tool_call_id": call["id"]}
    )
final = model_with_tools.invoke(messages)
```

Half 2 — the same tool name for both versions, so only the description differs:

```
@tool("get_weather")
def vague_weather(city: str) -> str:
    """gets data"""
    return f"Sunny in {city}"
```

Write `precise_weather` the same way with a precise docstring. Another way to keep it fair: a factory `make_weather_tool(description)` that defines one `get_weather`, sets `get_weather.__doc__ = description`, and returns `tool(get_weather)`. Try finishing both halves yourself before looking at the full Solution.

Full solution: Show me the solution